Customer Churn Prediction

Assignment 2 — Flask app using the Assignment 1 trained model

Project summary

This project serves the tuned Assignment 1 Logistic Regression model through a Flask web form. The saved scikit-learn pipeline includes fitted imputation, scaling, and one-hot encoding, so the app uses the same preprocessing as training. The form accepts all 19 model features and shows a human-readable prediction with estimated churn probability.

Model and evaluation

Item	Result
Selected model	Logistic Regression; tuned C = 30; no class weighting
Training-selected threshold	0.55
5-fold training CV accuracy	80.37%
Stratified holdout accuracy	79.77%
Holdout churn precision / recall / F1	67.05% / 46.79% / 55.12%
Holdout ROC AUC	0.8409
Majority-class baseline	73.46%

The requested accuracy above 85% was not achieved. The holdout set was not used to select the model or threshold; the result is reported as measured.

Project structure

```
project/
  app.py
  model.joblib
  requirements.txt
  churn_prediction.ipynb
  WA_Fn-UseC_-Telco-Customer-Churn (1).csv
  templates/index.html
  static/style.css
  tests/test_app.py
```

Implementation and verification

Model export

The notebook saves the tuned model pipeline, decision threshold, model name, and expected feature order as model.joblib.

```
model_bundle = {
    'model': best_model,
    'threshold': float(best_threshold),
    'model_name': str(best_model_name),
    'feature_names': X.columns.tolist(),
}
joblib.dump(model_bundle, 'model.joblib')
```

Flask app and interface

app.py loads the saved pipeline, checks its features against the form, validates numeric ranges and categorical options, and predicts from a DataFrame in training feature order. templates/index.html provides labeled fields and result/error messages; static/style.css provides the responsive layout.

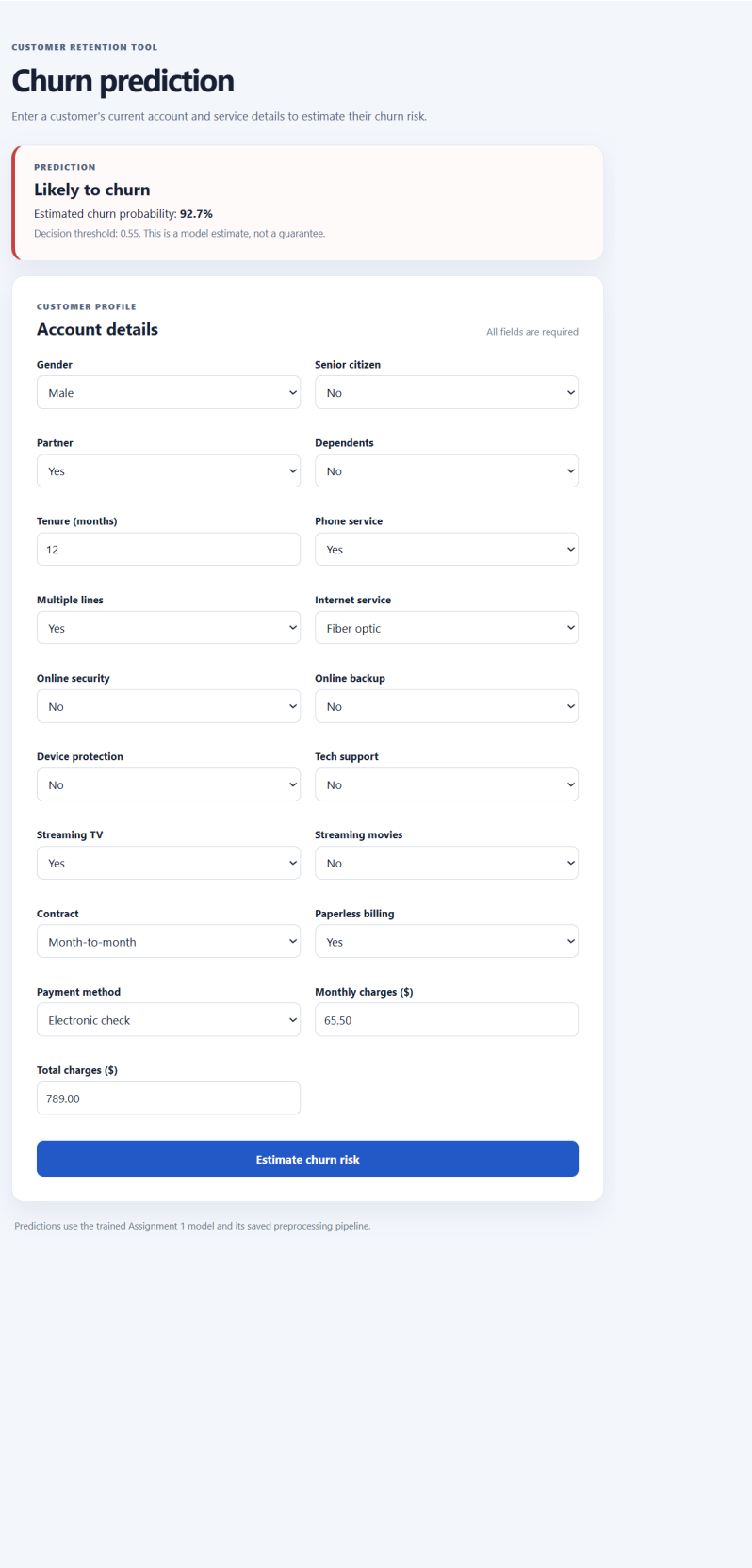
Seven unit tests passed: home page, expected churn, expected no-churn, missing input, invalid numeric range, unsupported category, and inconsistent phone-service options. Browser submissions confirmed both example predictions.

Local run: install requirements.txt, execute the notebook through the model-export cell, run python app.py, then open http://127.0.0.1:5000.

PythonAnywhere deployment is not complete because it requires the account owner’s configuration and public URL. Upload the app, model, requirements, templates, and static files; configure WSGI to import app as application; reload and test from another browser/device. Add the live URL and deployment screenshots after that is done.

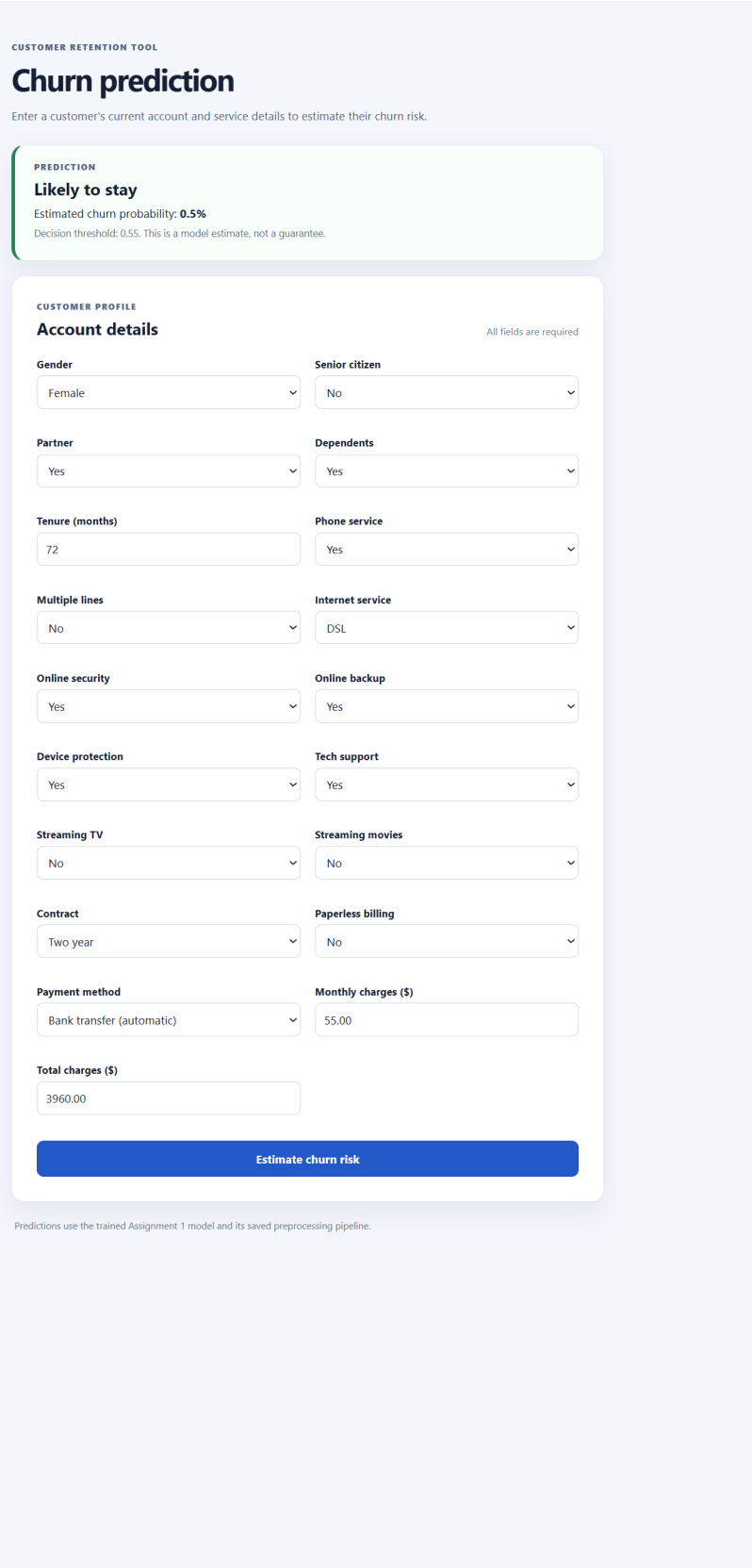
The model artifact is tied to the pinned scikit-learn version in requirements.txt.

Browser test — expected churn



Short-tenure, month-to-month fiber profile: likely to churn (92.7% estimated probability).

Browser test — expected no churn



Long-tenure customer on a two-year contract: likely to stay (0.5% estimated churn probability).